เอกสารประกอบการสอน **psql** และการจัดการสิทธิ์ **PostgreSQL**

1 ข้อมูลและบัญชีที่ใช้ในบทเรียน

postgres เป็นผู้ดูแลระบบ ใช้เตรียมสิทธิ์ระดับฐานข้อมูล ส่วน teacher เป็นผู้สร้างตาราง และ student01 เป็นผู้รับสิทธิ์ใช้งานตารางใน teacher_db

ใช้ฐานข้อมูลสำหรับการฝึกเท่านั้น หากมี teacher_db, teacher หรือ student01 อยู่แล้ว ให้ตรวจเจ้าของและสิทธิ์เดิมก่อน ผู้สอนไม่ควรสร้างซ้ำหรือลบของเดิมเพื่อเริ่มบทเรียน

รหัสผ่านให้ผู้เรียนกำหนดเอง ไม่เขียนรหัสผ่านจริงลงเอกสารหรือภาพหลักฐาน ตัวอย่าง SQL และคำสั่ง psql ในเอกสารนี้พิมพ์ได้โดยไม่ต้องคัดลอกข้อความ prompt

2 เริ่มเปิดใช้งานและเข้าสู่ระบบ

ขั้นที่ 1 ตรวจว่าติดตั้ง PostgreSQL แล้ว เปิด Start ของ Windows ค้นหา SQL Shell (psql) และเปิดโปรแกรม

ขั้นที่ 2 กรอกข้อมูลด้านล่าง ค่าภายในวงเล็บเหลี่ยมคือค่าเริ่มต้น กด Enter เพื่อใช้ค่านั้น

Server [localhost]: กด Enter

Database [postgres]: กด Enter

Port [5432]: กด Enter

Username [postgres]: กด Enter

Password for user postgres: พิมพ์รหัสผ่านของ postgres

ขณะพิมพ์รหัสผ่านจะไม่แสดงตัวอักษรหรือดอกจัน แล้วกด Enter รหัสนี้เป็นรหัสของบัญชี PostgreSQL ไม่ใช่รหัสเข้า Windows หรือรหัสหลักของ pgAdmin

ขั้นที่ 3 ตรวจสอบการเชื่อมต่อ

```
\conninfo
SELECT current_user, current_database();
```

ผลที่คาดหวัง: current_user เป็น postgres และ current_database เป็น postgres ส่วนพอร์ตในรายละเอียดการเชื่อมต่อเป็น 5432 หากติดตั้งด้วยค่าอื่นให้ใช้ค่าจริงของเครื่อง

ทางเลือกสำหรับเปิดจาก PowerShell

กรณีไม่ได้เพิ่ม psql ลง PATH ให้ใช้ตำแหน่งโปรแกรมโดยตรง พิมพ์บรรทัดนี้ใน PowerShell ไม่ใช่ในหน้าจอ psql

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" `
  -h localhost -p 5432 -U postgres -d postgres -W
```

เครื่องหมาย ` ท้ายบรรทัดใช้ต่อคำสั่ง PowerShell ส่วน -U ระบุผู้ใช้ -d ระบุฐานข้อมูล และ -W ให้ถามรหัสผ่าน

บันทึกผลและตอบคำถาม

บันทึก ชื่อผู้ใช้ ฐานข้อมูล server และ port ที่เชื่อมต่อจริง

คำถาม ชื่อ postgres ในช่อง Database และ Username หมายถึงสิ่งเดียวกันหรือไม่

ผลที่ได้ / คำตอบ .....

.....

3 อ่าน prompt ของ psql เปลี่ยนบัญชี และ ออกจากระบบ

ข้อความที่เห็น	ความหมาย
teacher_db=>	อยู่ฐานข้อมูล teacher_db โดยทั่วไปเป็นบัญชีที่ไม่ใช่ superuser
teacher_db=#	อยู่ฐานข้อมูล teacher_db โดยทั่วไปเป็น superuser
teacher_db(>	ยังมีวงเล็บเปิดค้างอยู่ รอพิมพ์คำสั่งให้ครบ
teacher_db'>	ยังมีข้อความใน single quote ที่ปิดไม่ครบ

prompt ของ psql สามารถปรับแต่งได้ จึงควรใช้คำสั่งตรวจบัญชีเพื่อยืนยันเสมอ

SELECT session_user, current_user, current_database();

session_user คือบัญชีที่เริ่ม session ส่วน current_user คือบัญชีที่ใช้ตรวจสิทธิ์ขณะนั้น อาจต่างกันเมื่อใช้ SET ROLE

เปลี่ยนการเชื่อมต่อเมื่อมีบัญชีและฐานข้อมูลแล้ว

\c teacher_db teacher localhost 5432
\conninfo

พิมพ์รหัสผ่านของ teacher เมื่อระบบถาม ถ้าเชื่อมต่อล้มเหลวและขึ้น Previous connection kept หมายถึง psql แบบโต้ตอบยังเก็บการเชื่อมต่อเดิมไว้ ต้องตรวจบัญชีอีกครั้ง

ยกเลิกคำสั่งที่พิมพ์ค้างและออกจากระบบ

กด Ctrl+C เพื่อยกเลิกข้อความที่กำลังพิมพ์ หรือยกเลิกคำสั่งที่กำลังทำงาน จากนั้นเริ่มพิมพ์ใหม่ ใช้ \r ล้าง query buffer เมื่อเรียก meta-command ได้

\q

\q ปิดการเชื่อมต่อและออกจาก psql ส่วน \c เปลี่ยนการเชื่อมต่อโดยยังอยู่ในโปรแกรม SQL ลงท้ายด้วย ; แต่คำสั่งที่ขึ้นต้นด้วย \ ไม่ต้องมี ;

บันทึกผลและตอบคำถาม

บันทึก บัญชีก่อนและหลังเปลี่ยนการเชื่อมต่อ และผลการออกแล้วเข้าใหม่

คำถาม เมื่อเปลี่ยนบัญชีไม่สำเร็จ ควรรัน GRANT ต่อทันทีหรือไม่ เพราะอะไร

ผลที่ได้ / คำตอบ .....

.....

4 แสดงฐานข้อมูลและบัญชีทั้งหมด

ขั้นที่ 1 เข้าระบบด้วย postgres แล้วทดลองคำสั่งทีละบรรทัด

คำสั่ง psql	สิ่งที่แสดง
\l หรือ \list	รายชื่อฐานข้อมูลทั้งหมดใน server ที่เชื่อมต่อ
\l+	รายละเอียดฐานข้อมูลเพิ่มเติม เช่น ขนาดที่ดูได้
\l teacher_db	รายละเอียดเฉพาะ teacher_db
\du	รายชื่อ role และคุณสมบัติ
\du+ teacher	รายละเอียด role teacher เพิ่มเติม
\drg	สมาชิกของ role และตัวเลือกสมาชิกใน PostgreSQL 18
\conninfo	รายละเอียดการเชื่อมต่อปัจจุบัน

ขั้นที่ 2 ทดลอง SQL สำหรับดูรายชื่อฐานข้อมูลพร้อมเจ้าของ

```
SELECT datname AS database_name,
       pg_get_userbyid(datdba) AS owner
FROM pg_database
ORDER BY datname;
```

ขั้นที่ 3 ทดลองดูคุณสมบัติของบัญชี โดยไม่แสดงข้อมูลรหัสผ่าน

```
SELECT rolname, rolcanlogin, rolsuper,
       rolcreatedb, rolcreaterole
FROM pg_roles
ORDER BY rolname;
```

ค่า t คือจริง f คือเท็จ role ที่มี LOGIN ใช้เข้าสู่ระบบได้ ส่วน role ที่ไม่มี LOGIN ใช้เป็นกลุ่มสิทธิ์ได้ การมองเห็นชื่อฐานข้อมูลไม่ได้แปลว่ามีสิทธิ์เชื่อมต่อ

บันทึกผลและตอบคำถาม

บันทึก ชื่อฐานข้อมูลอย่างน้อย 3 รายการ และเปรียบเทียบ role สองบัญชี

คำถาม role ที่ไม่มี LOGIN มีประโยชน์อย่างไร

ผลที่ได้ / คำตอบ .....

.....

5 คำสั่งตรวจตารางและโครงสร้างที่ควรรู้

เชื่อมต่อฐานข้อมูลที่ต้องการก่อน เพราะคำสั่งดูตารางทำงานภายในฐานข้อมูลปัจจุบัน
ตัวอย่างต่อไปนี้ใช้หลังสร้างตารางในหัวข้อ 8 แล้ว

คำสั่ง	สิ่งที่ตรวจสอบ
\dn+	schema เจ้าของ และสิทธิ์ของ schema
\dt	ตารางที่มองเห็นตาม search_path
\dt public.*	ตารางใน schema public
\dt *.*	ตารางทุก schema รวมตารางระบบ อาจมีจำนวนมาก
\dt+ public.*	ตารางพร้อมรายละเอียด เช่น ขนาด
\d public.test_students	คอลัมน์ ชนิดข้อมูล ค่าเริ่มต้น คีย์ และข้อกำหนด
\d+ public.test_students	โครงสร้างและรายละเอียดเพิ่มเติม
\di public.*	ดัชนีใน public
\ds public.*	sequence ใน public
\dv public.*	view ใน public
\dm public.*	materialized view ใน public
\df public.*	ฟังก์ชันใน public
\dp public.* หรือ \z public.*	รายการสิทธิ์ตาราง view และ sequence
\ddp	สิทธิ์เริ่มต้นสำหรับวัตถุที่จะสร้างใหม่
\? และ \h GRANT	วิธีใช้ meta-command และไวยากรณ์ SQL GRANT

ใช้ SHOW search_path; ตรวจลำดับ schema ที่ระบบค้นหาเมื่อไม่ระบุ schema หาก \dt ไม่พบตาราง ให้ลอง \dt public.* และตรวจฐานข้อมูลปัจจุบัน

บันทึกผลและตอบคำถาม

บันทึก ผล \dt และ \d โดยระบุ primary key และชนิดข้อมูลแต่ละคอลัมน์

คำถาม การดูรายชื่อตารางได้ ยืนยันว่า SELECT ข้อมูลได้เสมอหรือไม่

ผลที่ได้ / คำตอบ .....

.....

6 ทำความเข้าใจสิทธิ์แต่ละระดับ

ระดับ	สิทธิ์สำคัญ	ตัวอย่างผลที่อนุญาต
Role	LOGIN / CREATEDB / CREATEROLE	เข้าสู่ระบบ / สร้างฐานข้อมูล / จัดการ role ตามเงื่อนไข
Database	CONNECT / TEMPORARY / CREATE	เชื่อมต่อ / สร้างตารางชั่วคราว / สร้าง schema
Schema	USAGE / CREATE	อ้างถึงวัตถุ / สร้างวัตถุใน schema
Table	SELECT / INSERT / UPDATE / DELETE	อ่าน / เพิ่ม / แก้ไข / ลบแถว
Sequence	USAGE / SELECT / UPDATE	ใช้ nextval / อ่าน currval / ใช้ setval ตามสิทธิ์

การใช้งานตารางทั่วไปต้องผ่านสิทธิ์เชื่อมต่อฐานข้อมูล สิทธิ์ USAGE ของ schema และสิทธิ์ของ ตารางนั้น การให้ ALL PRIVILEGES ON DATABASE ไม่ได้ให้สิทธิ์ทุกตารางภายในโดยอัตโนมัติ

ผู้ให้สิทธิ์ต้องเป็นเจ้าของวัตถุ superuser หรือมีสิทธิ์นั้นพร้อม GRANT OPTION รวมถึงกรณีได้รับอำนาจผ่าน role ที่เกี่ยวข้อง เจ้าของฐานข้อมูลไม่จำเป็นต้องเป็นเจ้าของทุกตารางในฐานข้อมูล

PUBLIC หมายถึงทุก role รวมบัญชีที่จะสร้างภายหลัง ไม่ใช่ schema public โดยค่าเริ่มต้นฐานข้อมูลให้ CONNECT และ TEMPORARY แก่ PUBLIC

การถอนสิทธิ์จากบัญชีโดยตรงไม่ลบสิทธิ์ที่บัญชียังได้รับจาก PUBLIC กลุ่ม role หรือความเป็นเจ้าของ จึงต้องตรวจสิทธิ์ที่มีผลจริงควบคู่กัน

ตัวอย่างอาการที่ต้องแยกให้ออก

permission denied for database ⟶ ตรวจ CONNECT หรืออำนาจผู้ให้สิทธิ์
permission denied for schema public ⟶ ตรวจ USAGE และ CREATE
permission denied for table ⟶ ตรวจสิทธิ์ของตาราง
permission denied for sequence ⟶ ตรวจ sequence ที่ใช้สร้างรหัส

บันทึกผลและตอบคำถาม

บันทึก อธิบายสิทธิ์ที่จำเป็นในการสร้างตารางและอ่านตารางของผู้อื่น

คำถาม ทำไม GRANT CONNECT จึงยังไม่ทำให้ CREATE TABLE ได้

ผลที่ได้ / คำตอบ .....

.....

7 สร้างบัญชีและฐานข้อมูลสำหรับฝึก

ขั้นที่ 1 ใช้ postgres ตรวจว่ามีชื่อเหล่านี้แล้วหรือไม่ ถ้ามีให้ข้ามคำสั่ง CREATE ที่ซ้ำ และตรวจสิทธิ์เดิมกับผู้สอน

```
\du teacher
\du student01
\l teacher_db
```

ขั้นที่ 2 สร้างบัญชีใหม่ แล้วใช้ \password ตั้งรหัสผ่านทีละบัญชี

```
CREATE ROLE teacher LOGIN
  NOSUPERUSER NOCREATEDB NOCREATEROLE;
CREATE ROLE student01 LOGIN
  NOSUPERUSER NOCREATEDB NOCREATEROLE;
\password teacher
\password student01
```

ขั้นที่ 3 สร้างฐานข้อมูลใหม่ให้ postgres เป็นเจ้าของ และจำกัดสิทธิ์ส่วนรวม เฉพาะฐานข้อมูลที่ใช้ฝึกนี้

```
CREATE DATABASE teacher_db OWNER postgres;
REVOKE CONNECT, TEMPORARY
  ON DATABASE teacher_db FROM PUBLIC;
GRANT CONNECT, TEMPORARY
  ON DATABASE teacher_db TO teacher;
GRANT CONNECT
  ON DATABASE teacher_db TO student01;
```

teacher ได้สิทธิ์เชื่อมต่อและสร้างตารางชั่วคราว ส่วน student01 ได้สิทธิ์เชื่อมต่อ การถอนจาก PUBLIC ไม่ตัด session ที่เปิดไว้ และไม่ใช่การปิดสิทธิ์ของ superuser หรือเจ้าของ

หากต้องการให้ teacher เป็นเจ้าของฐานข้อมูลตั้งแต่ต้น ใช้ CREATE DATABASE ชื่อใหม่ OWNER teacher; เป็นอีกแบบหนึ่งของการจัดสิทธิ์ ไม่ต้องรันซ้ำกับ teacher_db ในบทนี้

บันทึกผลและตอบคำถาม

บันทึก ผล CREATE ROLE, CREATE DATABASE, REVOKE และ GRANT

คำถาม เหตุใดจึงถอนสิทธิ์ PUBLIC ก่อนให้สิทธิ์เฉพาะบัญชี

ผลที่ได้ / คำตอบ .....

.....

8 ให้สิทธิ์ schema และสร้างตาราง

ขั้นที่ 1 ใช้ postgres เชื่อมต่อ teacher_db แล้วให้สิทธิ์ของ schema

```
\c teacher_db postgres
GRANT USAGE, CREATE ON SCHEMA public TO teacher;
GRANT USAGE ON SCHEMA public TO student01;
```

ขั้นที่ 2 เปลี่ยนเป็น teacher ตรวจบัญชี แล้วสร้างตาราง

```
\c teacher_db teacher
SELECT current_user, current_database();
CREATE TABLE public.test_students (
  id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name text NOT NULL
);
```

ขั้นที่ 3 เพิ่มข้อมูลตั้งต้นและดูโครงสร้าง

```
INSERT INTO public.test_students (name)
VALUES ('สมชาย'), ('สมหญิง') RETURNING *;
\dt public.*
\d public.test_students
SELECT * FROM public.test_students ORDER BY id;
```

ผลที่คาดหวังเมื่อสร้างใหม่: ได้ CREATE TABLE และข้อมูลสองแถว id เป็น 1 และ 2 teacher เป็นเจ้าของตาราง จึงจัดการข้อมูลของตารางนี้ได้

หาก CREATE TABLE ล้มเหลว ห้ามทำ INSERT ต่อ ให้ตรวจสิทธิ์ schema ก่อน ส่วน table already exists หมายถึงตารางมีแล้ว ไม่ควรลบทิ้งโดยไม่ตรวจข้อมูล

บันทึกผลและตอบคำถาม

บันทึก บัญชีผู้สร้าง ชื่อ schema คอลัมน์ primary key และข้อมูลสองแถว

คำถาม student01 มี USAGE แต่ไม่มี CREATE จะสร้างตารางใน public ได้หรือไม่

ผลที่ได้ / คำตอบ .....

.....

9 ให้สิทธิ์อ่านตารางและทดสอบด้วยอีกบัญชี

ขั้นที่ 1 เปลี่ยนเป็น student01 แล้วลองอ่านก่อนให้สิทธิ์ตาราง

\c teacher_db student01
SELECT * FROM public.test_students;

ผลที่คาดหวังสำหรับบัญชีใหม่: permission denied for table test_students
บันทึกข้อผิดพลาดนี้เป็นผลการทดลอง ไม่ใช่รีบเพิ่มทุกสิทธิ์

ขั้นที่ 2 เปลี่ยนเป็น teacher ซึ่งเป็นเจ้าของตาราง แล้วให้สิทธิ์อ่าน

\c teacher_db teacher
GRANT SELECT ON TABLE public.test_students TO student01;
\dp public.test_students

ขั้นที่ 3 กลับเป็น student01 และทดลองอีกครั้ง

\c teacher_db student01
SELECT * FROM public.test_students ORDER BY id;

ผลที่คาดหวัง: อ่านได้สองแถว จากนั้นลองคำสั่งต่อไปนี้เพื่อยืนยันว่ายังเพิ่มข้อมูลไม่ได้

INSERT INTO public.test_students (name)
VALUES ('ทดสอบสิทธิ์') RETURNING *;

ผลที่คาดหวัง: permission denied for table เพราะ SELECT ไม่ได้รวม INSERT หากเพิ่มสำเร็จ แสดงว่าบัญชีมีสิทธิ์จากทางอื่น ต้องตรวจ PUBLIC กลุ่ม role และเจ้าของ

บันทึกผลและตอบคำถาม

บันทึก ผล SELECT ก่อนและหลัง GRANT และผล INSERT พร้อมชื่อบัญชี

คำถาม เหตุใดจึงต้องทดสอบสิทธิ์ด้วย student01 ไม่ใช่ postgres

ผลที่ได้ / คำตอบ .....

.....

10 เพิ่มสิทธิ์แก้ไขข้อมูลและถอนสิทธิ์บางส่วน

ขั้นที่ 1 ใช้ teacher ให้สิทธิ์ตารางที่ตนเองเป็นเจ้าของ

```
\c teacher_db teacher
GRANT INSERT, UPDATE, DELETE
  ON TABLE public.test_students TO student01;
```

ขั้นที่ 2 ใช้ student01 ทดลองทำรายการใน transaction เพื่อย้อนคืนข้อมูลได้

```
\c teacher_db student01
BEGIN;
INSERT INTO public.test_students (name)
VALUES ('ทดลองเพิ่ม') RETURNING *;
UPDATE public.test_students
SET name = 'ทดลองแก้ไข' WHERE id = 1 RETURNING *;
DELETE FROM public.test_students
WHERE id = 2 RETURNING *;
SELECT * FROM public.test_students ORDER BY id;
ROLLBACK;
```

หลัง ROLLBACK ข้อมูลแถวเดิมกลับคืน แต่ค่าที่ sequence ใช้แล้วอาจไม่ย้อนกลับ เลข id จึงไม่จำเป็นต้องต่อเนื่องทุกค่า

ขั้นที่ 3 ใช้ teacher ถอน DELETE แล้วทดสอบด้วย student01

```
\c teacher_db teacher
REVOKE DELETE ON TABLE public.test_students
  FROM student01;
\c teacher_db student01
DELETE FROM public.test_students WHERE id = 2;
```

ผลที่คาดหวัง: DELETE ถูกปฏิเสธ แต่ SELECT ยังทำได้ UPDATE ที่ใช้ WHERE หรือ RETURNING อาจต้องมี SELECT ของคอลัมน์ที่อ่านด้วย บทนี้ให้ SELECT ไว้แล้ว

บันทึกผลและตอบคำถาม

บันทึก จำนวนแถวที่เพิ่ม แก้ไข ลบ ผลหลัง ROLLBACK และผลหลังถอน DELETE

คำถาม REVOKE DELETE ทำให้ SELECT ถูกถอนด้วยหรือไม่

ผลที่ได้ / คำตอบ .....

.....

11 สิทธิ์ sequence และสิทธิ์ระดับคอลัมน์

ตารางที่ใช้ serial จะเรียก nextval ของ sequence ผ่านค่าเริ่มต้น สิทธิ์ INSERT บนตารางไม่ได้ให้สิทธิ์ sequence นั้นอัตโนมัติ ตัวอย่างนี้แยกจาก identity ในตารางก่อนหน้า

ขั้นที่ 1 ใช้ teacher สร้างตาราง serial และให้สิทธิ์ตาราง

```
\c teacher_db teacher
CREATE TABLE public.serial_demo (
  id serial PRIMARY KEY,
  note text
);
GRANT SELECT, INSERT ON public.serial_demo TO student01;
```

ขั้นที่ 2 ใช้ student01 ลองเพิ่ม คาดว่าจะถูกปฏิเสธที่ sequence หากไม่มีสิทธิ์เดิม

```
\c teacher_db student01
INSERT INTO public.serial_demo (note) VALUES ('ทดลอง');
```

ขั้นที่ 3 ใช้ teacher ตรวจชื่อ sequence แล้วให้ USAGE ตามชื่อจริง

```
\c teacher_db teacher
SELECT pg_get_serial_sequence('public.serial_demo', 'id');
GRANT USAGE ON SEQUENCE public.serial_demo_id_seq
  TO student01;
\c teacher_db student01
INSERT INTO public.serial_demo (note)
VALUES ('ทดลอง') RETURNING *;
```

ผลที่คาดหวัง: เพิ่มได้ หาก sequence เคยถูกเปลี่ยนชื่อ ให้แทนชื่อใน GRANT ด้วยชื่อที่ตรวจพบ

ตัวอย่างจำกัดการแก้ไขเฉพาะคอลัมน์ ให้ teacher รันสองคำสั่งต่อไปนี้ ใช้ได้เมื่อไม่มีสิทธิ์ UPDATE ระดับตารางจากช่องทางอื่น

```
REVOKE UPDATE ON public.test_students FROM student01;
GRANT UPDATE (name) ON public.test_students TO student01;
```

บันทึกผลและตอบคำถาม

บันทึก ข้อผิดพลาดก่อนให้ USAGE และผลหลังให้สิทธิ์

คำถาม สิทธิ์ของ sequence กับสิทธิ์ INSERT ของตารางเป็นสิทธิ์เดียวกันหรือไม่

ผลที่ได้ / คำตอบ .....

.....

12 สิทธิ์ในการให้สิทธิ์ต่อและการเป็นเจ้าของ

ขั้นที่ 1 ใช้ postgres ให้ teacher ส่งต่อสิทธิ์ CONNECT ของฐานข้อมูลได้

```
\c teacher_db postgres
GRANT CONNECT ON DATABASE teacher_db
  TO teacher WITH GRANT OPTION;
```

ขั้นที่ 2 ใช้ teacher ทดลองให้สิทธิ์นั้นแก่บัญชีที่มีอยู่แล้ว

```
\c teacher_db teacher
GRANT CONNECT ON DATABASE teacher_db TO student01;
```

GRANT OPTION อนุญาตให้ส่งต่อเฉพาะสิทธิ์ที่ได้รับ ไม่ได้ทำให้ teacher เป็น superuser หรือสร้าง role ได้ teacher ให้สิทธิ์ของ test_students ได้อยู่แล้วเพราะเป็นเจ้าของตาราง

ขั้นที่ 3 ตัวอย่างถอนเฉพาะอำนาจให้สิทธิ์ต่อ โดย postgres
คำสั่งอาจถูกปฏิเสธหากมีสิทธิ์ที่ผู้อื่นได้รับต่อเนื่องอยู่

```
\c teacher_db postgres
REVOKE GRANT OPTION FOR CONNECT
  ON DATABASE teacher_db FROM teacher RESTRICT;
```

RESTRICT เป็นค่าเริ่มต้นและจะไม่ถอนต่อแบบลูกโซ่ หากต้องใช้ CASCADE
ต้องตรวจผู้รับสิทธิ์ต่อก่อน เพราะสิทธิ์ที่พึ่งพาการให้ต่ออาจถูกถอนด้วย ส่วนสิทธิ์ CONNECT ของ teacher เองยังคงอยู่

เจ้าของกับผู้ได้รับสิทธิ์ต่างกันอย่างไร

เจ้าของมีอำนาจจัดการวัตถุ เช่น ALTER หรือ DROP ซึ่งไม่ได้ให้ด้วย GRANT SELECT หรือ GRANT ALL เพียงอย่างเดียว ตัวอย่างต่อไปนี้เป็นทางเลือกสำหรับผู้ดูแล ไม่ต้องรันในลำดับฝึกหลัก

```
ALTER DATABASE teacher_db OWNER TO teacher;
ALTER TABLE public.test_students OWNER TO teacher;
```

สองคำสั่งเปลี่ยนเจ้าของคนละระดับ
เปลี่ยนเจ้าของฐานข้อมูลไม่ได้เปลี่ยนเจ้าของทุกตารางที่อยู่ภายใน

บันทึกผลและตอบคำถาม

บันทึก ผู้ให้สิทธิ์ ผู้รับสิทธิ์ และผล GRANT หรือ REVOKE ที่เกิดจริง

คำถาม WITH GRANT OPTION ต่างจากการเป็นเจ้าของวัตถุอย่างไร

ผลที่ได้ / คำตอบ .....

.....

13 รวมสิทธิ์ไว้ในกลุ่ม role

เมื่อมีผู้เรียนหลายบัญชี สามารถให้สิทธิ์กับ role กลาง แล้วเพิ่มบัญชีเป็นสมาชิก
ช่วยลดการให้สิทธิ์ซ้ำรายคน

ขั้นที่ 1 ใช้ postgres สร้าง role กลางและให้สิทธิ์พื้นฐาน

```
\c teacher_db postgres
CREATE ROLE class_reader NOLOGIN;
GRANT CONNECT ON DATABASE teacher_db TO class_reader;
GRANT USAGE ON SCHEMA public TO class_reader;
GRANT class_reader TO student01 WITH INHERIT TRUE;
```

ขั้นที่ 2 ใช้ teacher ให้กลุ่มอ่านตารางที่ตนเองเป็นเจ้าของ

```
\c teacher_db teacher
GRANT SELECT ON public.test_students TO class_reader;
```

ขั้นที่ 3 ใช้ student01 ตรวจสมาชิกและสิทธิ์ที่ได้รับ

```
\c teacher_db student01
\drg
SELECT pg_has_role(current_user,
                   'class_reader', 'USAGE') AS inherited;
SELECT * FROM public.test_students;
```

ผล inherited ควรเป็น t ในตัวอย่างนี้ WITH INHERIT TRUE ทำให้ใช้สิทธิ์สมาชิกได้โดยไม่ต้อง SET ROLE

ขั้นที่ 4 เมื่อต้องการยกเลิกสมาชิก ให้ postgres รัน

```
\c teacher_db postgres
REVOKE class_reader FROM student01;
```

student01 ยังอาจ SELECT ได้ เพราะได้รับ SELECT โดยตรงในหัวข้อ 9
การถอนสมาชิกไม่ได้ถอนสิทธิ์ที่ได้รับโดยตรง WITH ADMIN OPTION ของสมาชิก role เป็นคนละเรื่องกับ WITH GRANT OPTION ของสิทธิ์วัตถุ

บันทึกผลและตอบคำถาม

บันทึก สมาชิกก่อนและหลัง REVOKE และผล SELECT ที่ยังได้รับโดยตรง

คำถาม เหตุใดถอนสมาชิกแล้วจึงอาจยังอ่านตารางได้

ผลที่ได้ / คำตอบ .....

.....

14 ให้สิทธิ์ตารางปัจจุบันและตารางที่จะสร้างใหม่

ขั้นที่ 1 ใช้ teacher ให้ SELECT บนตารางที่มีอยู่แล้วใน public
คำสั่งนี้ครอบคลุมเฉพาะวัตถุที่ผู้สั่งมีอำนาจให้สิทธิ์

```
\c teacher_db teacher
GRANT SELECT ON ALL TABLES IN SCHEMA public
  TO student01;
```

ALL TABLES IN SCHEMA มีผลกับวัตถุปัจจุบัน ไม่ได้ตั้งให้ตารางใหม่ได้รับสิทธิ์อัตโนมัติ
ขั้นที่ 2 ตั้งสิทธิ์เริ่มต้นของตารางใหม่ที่ teacher จะสร้างใน public

```
ALTER DEFAULT PRIVILEGES FOR ROLE teacher
  IN SCHEMA public
  GRANT SELECT ON TABLES TO student01;
\ddp
```

ขั้นที่ 3 ยังใช้ teacher สร้างตารางใหม่ แล้วใช้ student01 อ่าน

```
CREATE TABLE public.next_lesson (topic text);
INSERT INTO public.next_lesson VALUES ('psql');
\c teacher_db student01
SELECT * FROM public.next_lesson;
```

ผลที่คาดหวัง: อ่านได้หนึ่งแถวโดยไม่ต้อง GRANT เฉพาะตารางซ้ำ ข้อนี้ใช้กับตารางที่ teacher สร้างเท่านั้น ไม่ครอบคลุมตารางที่ postgres สร้าง และไม่ย้อนกลับไปแก้ตารางเก่า
ขั้นที่ 4 หากไม่ต้องการตั้งให้ตารางถัดไปอีก ให้ teacher รัน

```
\c teacher_db teacher
ALTER DEFAULT PRIVILEGES FOR ROLE teacher
  IN SCHEMA public
  REVOKE SELECT ON TABLES FROM student01;
```

การยกเลิกค่าเริ่มต้นไม่ถอน SELECT จาก next_lesson ที่สร้างไปแล้ว หากต้องการถอนต้อง REVOKE ที่ตารางนั้นเพิ่มเติม

บันทึกผลและตอบคำถาม

บันทึก ผล \ddp และผล SELECT ตารางใหม่

คำถาม ถ้า postgres สร้างตารางใหม่ ค่าเริ่มต้นของ teacher จะตามไปด้วยหรือไม่

ผลที่ได้ / คำตอบ .....

.....

15 เปลี่ยนรหัสผ่านและควบคุมการเข้าสู่ระบบ

ขั้นที่ 1 เปลี่ยนรหัสผ่านของตนเอง หลังเข้าสู่ระบบด้วย teacher

\c teacher_db teacher

\password

พิมพ์รหัสใหม่และยืนยันอีกครั้ง คำสั่งนี้หลีกเลี่ยงรหัสผ่านแบบข้อความธรรมดาในประวัติคำสั่ง ไม่ต้องเขียนรหัสจริงใน SQL ของใบงาน

ขั้นที่ 2 ออกจากโปรแกรมแล้วเปิด SQL Shell ใหม่ เลือก teacher_db และ teacher ทดลองเข้าสู่ระบบด้วยรหัสใหม่

\q

ขั้นที่ 3 กรณีผู้เรียนลืมรหัส ให้ผู้ดูแลที่เข้าสู่ระบบได้ใช้ postgres รีเซ็ตให้

\c teacher_db postgres

\password teacher

คำสั่งรีเซ็ตนี้ต้องใช้ session ที่มีอำนาจจัดการบัญชี หาก postgres เองลืมรหัสและไม่มีบัญชีผู้ดูแลอื่น ต้องใช้ขั้นตอนกู้คืนของผู้ดูแลเครื่อง เอกสารนี้ไม่ให้เปลี่ยนระบบยืนยันตัวตนเป็น trust สำหรับการฝึกทั่วไป

ทดลองระงับและเปิดการเข้าสู่ระบบด้วยผู้ดูแล

ALTER ROLE teacher NOLOGIN;

ALTER ROLE teacher LOGIN;

หากจะทดสอบ NOLOGIN ให้เปิดอีกหน้าต่างลองเข้า teacher ก่อนรัน LOGIN เพื่อเปิดคืน NOLOGIN ไม่ตัด session เดิมที่เชื่อมต่ออยู่

กำหนดวันหมดอายุของรหัสผ่านเมื่อจำเป็น

ALTER ROLE teacher VALID UNTIL '2027-12-31 23:59:59+07';

ALTER ROLE teacher VALID UNTIL 'infinity';

คำสั่งแรกกำหนดวันหมดอายุสำหรับการยืนยันตัวตนด้วยรหัสผ่าน คำสั่งหลังยกเลิกวันหมดอายุ ไม่ใช่การลบบัญชีหรือถอนสิทธิ์ตาราง

บันทึกผลและตอบคำถาม

บันทึก ผลเข้าใหม่หลังเปลี่ยนรหัส และผลทดลอง NOLOGIN โดยไม่บันทึกรหัส

คำถาม การเปลี่ยนรหัสผ่านทำให้สิทธิ์ SELECT เดิมหายไปหรือไม่

ผลที่ได้ / คำตอบ .....

.....

16 ตรวจว่าบัญชีเข้าใช้ฐานข้อมูลใดได้

ขั้นที่ 1 ตรวจว่ากำลังตรวจบัญชีใด สามารถใช้ postgres ตรวจบัญชีอื่นด้วยชื่อ role ที่ระบุในฟังก์ชัน

```
SELECT session_user, current_user, current_database();
```

ขั้นที่ 2 แสดงสิทธิ์ของ teacher ต่อฐานข้อมูลทั้งหมดใน server ปัจจุบัน

```
SELECT d.datname AS database_name,
  pg_get_userbyid(d.datdba) AS owner,
  d.datallowconn AS allows_connections,
  has_database_privilege('teacher', d.oid,
                         'CONNECT') AS can_connect,
  has_database_privilege('teacher', d.oid,
                         'CREATE') AS can_create_schema,
  has_database_privilege('teacher', d.oid,
                         'TEMPORARY') AS can_create_temp
FROM pg_database AS d
ORDER BY d.datname;
```

เปลี่ยน teacher เป็น student01 เพื่อตรวจอีกบัญชี หรือใช้ current_user
โดยไม่ใส่อัญประกาศเพื่อตรวจบัญชีปัจจุบัน

ผล t ของ can_connect คือมีสิทธิ์ CONNECT ไม่รับประกันว่าเข้าสู่ระบบได้จริง ยังขึ้นกับ LOGIN รหัสผ่าน pg_hba.conf สถานะอนุญาตเชื่อมต่อ และข้อจำกัดการเชื่อมต่อของ server

ขั้นที่ 3 ตรวจสถานะของบัญชี

```
SELECT rolname, rolcanlogin, rolvaliduntil
FROM pg_roles
WHERE rolname IN ('teacher', 'student01');
```

ขั้นที่ 4 ทดลองเชื่อมต่อฐานข้อมูลที่ต้องการด้วยบัญชีนั้นและใช้ \conninfo ยืนยัน
ไม่ควรสรุปว่าบัญชีใช้ได้เฉพาะ teacher_db เพราะฐานข้อมูลอื่นอาจยังให้ CONNECT ผ่าน PUBLIC

บันทึกผลและตอบคำถาม

บันทึก ฐานข้อมูลที่ can_connect เป็น t พร้อม LOGIN และผลเข้าใช้งานจริง

คำถาม ทำไม can_connect เป็น t จึงยังอาจเข้าสู่ระบบไม่ได้

ผลที่ได้ / คำตอบ .....

.....

17 ตรวจว่าบัญชีใช้ตารางใดและทำอะไรได้

ขั้นที่ 1 เชื่อมต่อ teacher_db แล้วรัน SQL นี้เพื่อตรวจ student01
ตารางในฐานข้อมูลอื่นต้องเชื่อมต่อไปตรวจแยกทีละฐานข้อมูล

```
SELECT n.nspname AS schema_name,
  c.relname AS table_name,
  pg_get_userbyid(c.relowner) AS owner,
  has_schema_privilege('student01', n.oid,
                       'USAGE') AS schema_usage,
  has_table_privilege('student01', c.oid,
                      'SELECT') AS can_select,
  has_table_privilege('student01', c.oid,
                      'INSERT') AS can_insert,
  has_table_privilege('student01', c.oid,
                      'UPDATE') AS can_update,
  has_table_privilege('student01', c.oid,
                      'DELETE') AS can_delete
FROM pg_class AS c
JOIN pg_namespace AS n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r', 'p')
  AND n.nspname !~ '^pg_'
  AND n.nspname <> 'information_schema'
ORDER BY n.nspname, c.relname;
```

ผลแต่ละแถวแสดงสิทธิ์ระดับตาราง โดย r คือ ordinary table และ p คือ partitioned table
หากต้องการรวม view และ materialized view ให้เพิ่ม v และ m ในเงื่อนไข relkind

ต้องพิจารณา schema_usage ร่วมกับสิทธิ์ตาราง ตารางที่ SELECT เป็น t แต่ USAGE เป็น f ยังอ้างถึงผ่าน schema นั้นไม่ได้ตามปกติ

ฟังก์ชัน has_table_privilege รวมสิทธิ์ที่ใช้ได้จาก role และ PUBLIC
แต่ไม่สรุปสิทธิ์เฉพาะคอลัมน์ให้เป็นสิทธิ์ทั้งตาราง นอกจากนี้ Row Level Security อาจจำกัดแถว และ constraint อาจทำให้เขียนข้อมูลไม่สำเร็จ

บันทึกผลและตอบคำถาม

บันทึก เลือก 2 ตาราง ระบุบัญชีและสิทธิ์อ่าน เพิ่ม แก้ไข ลบที่ตรวจพบ

คำถาม ถ้า can_update เป็น f แต่แก้ name ได้ ต้องตรวจสิทธิ์ระดับใดต่อ

ผลที่ได้ / คำตอบ .....

.....

18 ตรวจสิทธิ์คอลัมน์ sequence และที่มาของสิทธิ์

ขั้นที่ 1 ตรวจสิทธิ์ UPDATE ของแต่ละคอลัมน์ กรณีให้ UPDATE (name) ไว้

```
SELECT a.attname AS column_name,
  has_column_privilege('student01', a.attrelid,
    a.attnum, 'UPDATE') AS can_update
FROM pg_attribute AS a
WHERE a.attrelid = 'public.test_students'::regclass
  AND a.attnum > 0 AND NOT a.attisdropped
ORDER BY a.attnum;
```

ขั้นที่ 2 ตรวจ sequence ที่ใช้กับ serial_demo

```
SELECT has_sequence_privilege('student01',
  'public.serial_demo_id_seq', 'USAGE') AS can_use_seq;
```

ขั้นที่ 3 ดูรายการสิทธิ์และค่าเริ่มต้นเพื่อหาที่มาของสิทธิ์

```
\l teacher_db
\dn+ public
\dp public.test_students
\dp public.serial_demo_id_seq
\drg
\ddp
```

รูปแบบรายการสิทธิ์มักเป็น ผู้รับ=รหัสสิทธิ์/ผู้ให้ เช่น student01=r/teacher หมายถึง teacher ให้ SELECT แก่ student01 ถ้าหน้าสัญลักษณ์ = ว่างหมายถึง PUBLIC เครื่องหมาย * หลังรหัสคือมี GRANT OPTION

รหัสที่พบบ่อย: r คือ SELECT, a คือ INSERT, w คือ UPDATE, d คือ DELETE, D คือ TRUNCATE, x คือ REFERENCES, t คือ TRIGGER ส่วน C คือ CREATE, c คือ CONNECT, T คือ TEMPORARY และ U คือ USAGE ต้องอ่านตามชนิดวัตถุ

ช่อง Access privileges ว่างอาจหมายถึงใช้ค่าเริ่มต้น ไม่ใช่ไม่มีสิทธิ์ทั้งหมด รายการ ACL ไม่ใช่รายงานสิทธิ์ที่มีผลจริงครบทุกแหล่ง จึงใช้คู่กับ has_*_privilege และการทดสอบด้วยบัญชีจริง

บันทึกผลและตอบคำถาม

บันทึก ตัวอย่าง ACL หนึ่งรายการพร้อมคำแปล และผลสิทธิ์คอลัมน์ name

คำถาม ทำไมดูเฉพาะ \dp จึงอาจยังสรุปสิทธิ์ของบัญชีไม่ได้ครบ

ผลที่ได้ / คำตอบ .....

.....

19 บันทึกผลลัพธ์และเรียกใช้ไฟล์คำสั่ง

ขั้นที่ 1 สร้างโฟลเดอร์ C:/psql_lab ใน Windows สำหรับเก็บไฟล์ของตนเอง แล้วใน psql ใช้คำสั่งต่อไปนี้เพื่อบันทึกผลคำสั่ง

\pset pager off
\o C:/psql_lab/result.txt
SELECT current_user, current_database();
\conninfo
\dt public.*
SELECT * FROM public.test_students ORDER BY id;
\o

\o ที่ไม่มีชื่อไฟล์จะคืนผลออกหน้าจอ result.txt เก็บผลลัพธ์ที่ส่งไปยังช่องผลคำสั่ง ไม่ใช่บันทึกหน้าจอครบทุกอย่าง เช่น ข้อผิดพลาดอาจไม่อยู่ในไฟล์ ต้องเก็บข้อความ ERROR หรือภาพหน้าจอแยก

ขั้นที่ 2 ส่งออกข้อมูลเป็น CSV คำสั่ง \copy ต่อไปนี้ให้พิมพ์เป็นบรรทัดเดียว

\copy public.test_students TO 'C:/psql_lab/students.csv' CSV HEADER

\copy เขียนไฟล์ที่เครื่องซึ่งรัน psql ใช้สิทธิ์อ่านตารางและสิทธิ์เขียนโฟลเดอร์ของผู้ใช้ Windows ไม่ใช่การสั่ง server เขียนไฟล์ด้วย SQL COPY

ขั้นที่ 3 สร้างไฟล์ check.sql ด้วยโปรแกรมแก้ไขข้อความ บันทึก UTF-8 โดยใส่ SELECT current_user, current_database(); แล้วเรียกจาก psql

\i C:/psql_lab/check.sql

รูปแบบบันทึกที่ใช้ทุกการทดลอง

วันเวลา ..... บัญชี ..... ฐานข้อมูล .....
คำสั่งที่ทดลอง .....
ผลจริง / จำนวนแถว / ข้อความ ERROR .....
ผลที่คาดหวัง ..... ตรงตามคาดหรือไม่ .....
ชื่อไฟล์หลักฐาน ..... สาเหตุและวิธีแก้ .....

ตัวอย่าง: ใช้ student01 อ่าน test_students หลัง GRANT SELECT ได้ 2 แถว ตรงตามคาด หลักฐาน 09_select_student01.png คำตอบ: SELECT ให้อ่านข้อมูล แต่ยังไม่อนุญาตให้ INSERT